Turtlebot3 Autonomy Assignment (ROS 2 Humble)

Submission Format

Submit all code to a **public GitHub repository** with a clear and concise README.
Invite `@rphly` as a collaborator to the repository.

Target Platform

- **ROS 2 Humble**
- **Turtlebot3 Burger**
- All tasks must be simulated and demonstrable in **Gazebo**
- Navigation paths must be visualized using **Rviz**
- **Bonus:** If feasible, demonstrate the system on a physical Turtlebot3 Burger

---

Section 1: Exploration and SLAM

Objective

Implement autonomous exploration using ROS 2 Humble on the Turtlebot3 Burger. The robot should explore an unknown office environment, build a map using SLAM, and avoid obstacles.

Problem Statement

Develop a ROS 2 Humble-based exploration stack that enables the Turtlebot3 Burger to autonomously navigate and create a 2D occupancy grid map of an office environment using SLAM. The robot must avoid collisions with walls and obstacles while maximizing map coverage.

Deliverables

- ROS 2 launch files for SLAM and navigation.
- Use `slam_toolbox` or a similar ROS 2-compatible SLAM package.
- Rviz configuration to visualize mapping and robot state.
- A saved map (`.pgm` and `.yaml`).
- A video or simulation recording showing successful exploration and obstacle avoidance.

Section 2: Agentic Semantic Reasoning (Bonus + Simulation)

Objective

Design and simulate a system where the robot is capable of associating semantic meaning to locations it observes during exploration, mimicking human-like memory and reasoning.

Problem Statement

As a robot explores its environment, it should be able to build not only a spatial map but also semantic context — similar to how a human remembers where the "toilet" or "pantry" is after walking through an office.

You are asked to design and implement a system that allows the robot to **semantically tag its environment** using onboard sensors (e.g., camera, depth) and retrieve these locations later in response to a user query.

You may use or simulate tools such as vision-language models (e.g., OpenAI APIs, CLIP, etc.), but the core value lies in **how you design the architecture** and manage semantic data over space.

Open Guidelines

- Propose a system for capturing, storing, and retrieving semantic location information.
- Demonstrate your robot identifying a location (e.g., "this is the meeting room") and later returning to that location based on a text query.
- Semantic queries should not be hard-coded. Instead, demonstrate a general framework or API that accepts a text prompt and returns a path to the corresponding place.
- You may simulate or mock the actual vision-language model outputs for this assignment, but you must clearly describe how real integration would work.

Deliverables

- A clear explanation of your system design: how semantic labels are generated, stored, and queried.
- A working ROS 2 node or system that demonstrates this flow in simulation.
- Simulated or mocked examples of image-to-label tagging (e.g., "this image contains a toilet").
- A short test case where a semantic query (e.g., “Where is the toilet?”) results in navigation to the correct location.

Section 3: Planner Task (Revised)

Objective

Implement a custom RRT (Rapidly-Exploring Random Tree) planner node that generates a path on a *provided or prebuilt* 2D occupancy grid map.

Problem Statement

Instead of converting a `.png` floor plan, use **any existing demo map or playground** (e.g., a TurtleBot3/Gazebo world, Nav2 sample maps, or a map you created in Section 1). Obtain an occupancy grid and implement an RRT planner that computes a path between a specified start pose and goal pose.

> You may:
>
> - Load a map via `nav2_map_server` (YAML + image pair),
> - Or build a map quickly with `slam_toolbox` in a demo world, then save it and use it as a static map.

Inputs to the RRT Planner Node

- Occupancy grid (`nav_msgs/OccupancyGrid`)
- Start pose (`geometry_msgs/PoseStamped`)
- Goal pose (`geometry_msgs/PoseStamped`)

Requirements & Notes

- Publish the planned path as `nav_msgs/Path` and visualize in RViz.
- Clearly document any map source (demo world name, saved map path).
- RRT* or goal-biasing is optional but encouraged—document your choices.
- Ensure your node handles grid resolution, origin, and obstacle inflation assumptions.

Deliverables

- A ROS 2 node that implements the RRT path planner.
- RViz visualization of the generated path.
- Example run using a demo/playground map (include commands/launch files).
- (Optional) Simulation showing the robot following the planned path using your planner output.